

MySQL Time Travel

Extended version

MySQL tables in Parquet, five minutes behind. Every version of every row.

Daniel Guzman Burgos dbtrail

Fri 11 Sep, 15:45 Matterhorn 1

Deleted at 14:04.

Back by 14:06.

MYSQL

POSTGRES

BETA

DUCKDB

```
mysql> SELECT * FROM customers WHERE id = 4827;
Empty set (0.00 sec)

mysql> -- whoops. what was it at 14:04?
mysql> SELECT * FROM _flashback.customers
      ->   AS OF '2026-06-10 14:04:00' WHERE id = 4827;
+-----+-----+-----+
| 4827 | Acme Corp | gold |
+-----+-----+-----+
1 row in set (0.02 sec)
```

HEARD AT THIS CONFERENCE, WEDNESDAY

Everyone wants DuckDB next to MySQL.

OPEN SOURCE



app →

MYSQLD · FORK 8.0.44

InnoDB

DuckDB

Replays its own ROW binlog into columns and folds to one final row per key.

runs where you can run AliSQL

ALPHA, NOT IN RELEASES



app →

MARIADB · PLUGIN

InnoDB

DuckDB

Tables you create as ENGINE=DuckDB. InnoDB and DuckDB in one query.

runs where you can load an alpha plugin

EXPERIMENTAL



app →

MYSQLD 9.7 · IN TREE

InnoDB

DuckDB

A second mysqld fed by row-based replication. You create its tables as ENGINE=DuckDB by hand; row changes then land in DuckDB, and mysqld hands each query to it.

experimental, v0.2.3 · runs where you can build MySQL 9.7 from their tree

APACHE 2.0, SHIPPING



MYSQLD · STOCK

InnoDB

binlog →

dbtrail

→

DuckDB

over Parquet

Nothing inside the server. The Parquet written for recovery is the Parquet DuckDB reads.

runs on RDS and Aurora, where you cannot load an engine

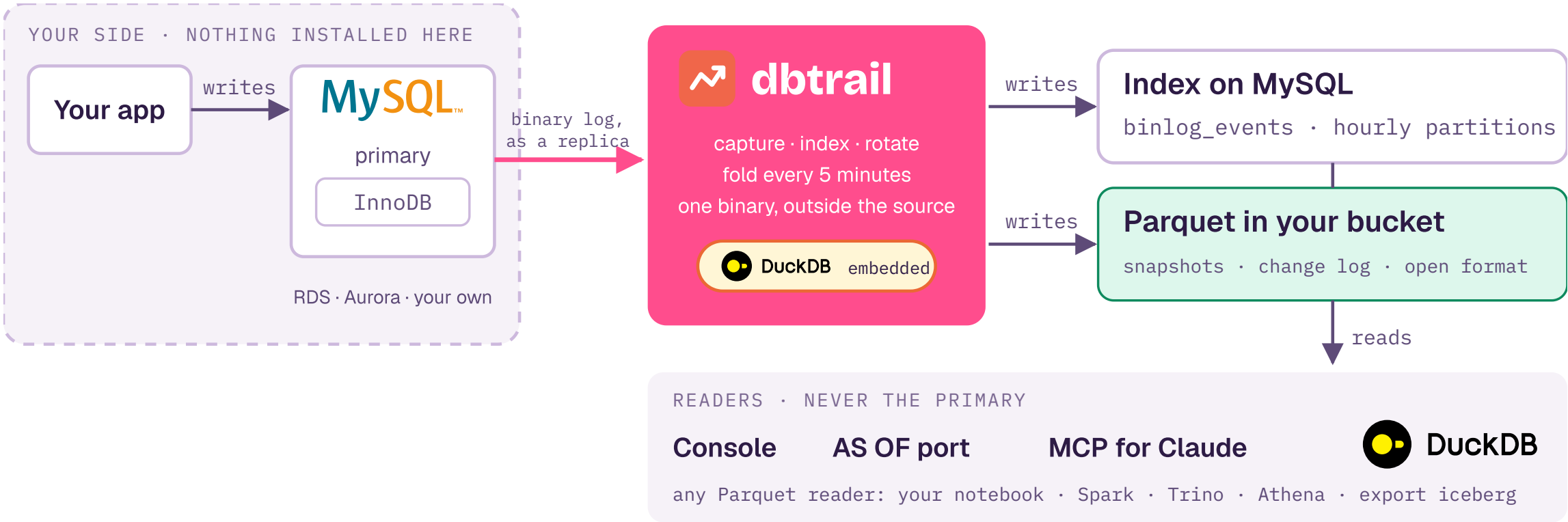
Three of the four change mysqld. Same idea, different place to put it.

WHAT IT IS

Your MySQL tables in Parquet, five minutes behind. Every version of every row.

- DBTrail connects to **RDS, Aurora or your own MySQL** the way a replica does: no agent, no binlog files on disk
- writes plain Parquet to **your bucket**
- query it with **DuckDB**, or anything that reads Parquet

```
Position:  mysql-bin-changelog.050309:529
Events:    69,298,207
Freshness: newest event is 8s old
```



The primary never sees a query from this box. Reports, time travel and the backup check all read the copy.
The files are plain Parquet: they open with nothing of ours running.

MYSQL 8 · MARIADB · PERCONA SERVER · RDS · AURORA
POSTGRESQL: CAPTURE AND ROW TIME TRAVEL IN BETA · THE REFRESHED COPY IS NEXT (#597)

WHAT THE FOURTH BOX MEANS

DuckDB outside MySQL. Five things that follow.

RESOURCES

MYSQLD

InnoDB

BUFFER POOL

100 % yours

DuckDB

own process

0 READS ACROSS

Nothing taken from InnoDB

Buffer pool, CPU and IOPS stay with your transactions.

UNBOUNDED TABLES

HOT TABLE

2024

2025

2026

purge

2024.parquet

2025.parquet

STAYS SMALL

DUCKDB - AS OF

STILL READ THEM

Old rows can leave InnoDB

Purge last year from MySQL; the copy and AS OF still have it.

DISK

ORDERS - 2.25 M ROWS

118 MB InnoDB

16.5 MB Parquet

CUSTOMERS - 2.25 M ROWS

276 MB

29 MB

7 TO 9× SMALLER

7 to 9× smaller on disk

Columnar and compressed; measured on the demo tables.

FRESHNESS

every 5 min

change log

fold

next snapshot

0 READS ON PRIMARY

Refreshed every 5 minutes

A fold of the change log, never a dump; on your schedule.

WHERE IT RUNS

RDS

Aurora

MySQL 8.0

DuckDB

same tables

NO MIGRATION
A REPLICA GRANT

RDS, Aurora, plain MySQL

No engine to load, no fork, no migration.

Three of the four put DuckDB inside mysql. The fourth gives DuckDB to the MySQL you cannot change.

WHAT YOU DO WITH IT

Four things you do with the copy.

REPORTS

MYSQLD · APP

InnoDB

POOL · CPU · IOPS

100 % yours

DuckDB

3 joins · GROUP BY

2.25 M rows · 0.5 s

own process

0 CONTENTION

The ugly query runs somewhere else

Not inside mysqld. A separate process, over the copy. It never competes with your transactions for buffer pool, CPU or IOPS.

15 to 19 s on RDS · 0.5 s on the copy · primary did 0 work

REPLICA

RDS READ REPLICA

MYSQLD · AGAIN

InnoDB

same 18.7 s

a second bill to patch

YOUR BUCKET

orders.parquet

customers.parquet

0.5 s · 7 to 9× smaller

NO SERVER

Retire the reporting replica

A read replica is another MySQL to pay for and patch, with the same query time. This is files in a bucket.

not a server to patch

TABLES

HOT TABLE

2024

2025

2026

STAYS SMALL

purge

2024.parquet

2025.parquet

DUCKDB · AS OF STILL READ THEM

Shrink InnoDB

Purge last year from the hot table. The copy and AS OF still have every row, at every date.

unbounded tables stay small

HISTORY

14:03

gold

14:04

UPDATE, no WHERE

undo SQL

backup ✓

VERIFIED · NO RESTORE

Undo and verify

The UPDATE without WHERE, the DELETE that cascaded: the SQL that reverses it. And the backup checks itself, no restore.

every version of every row

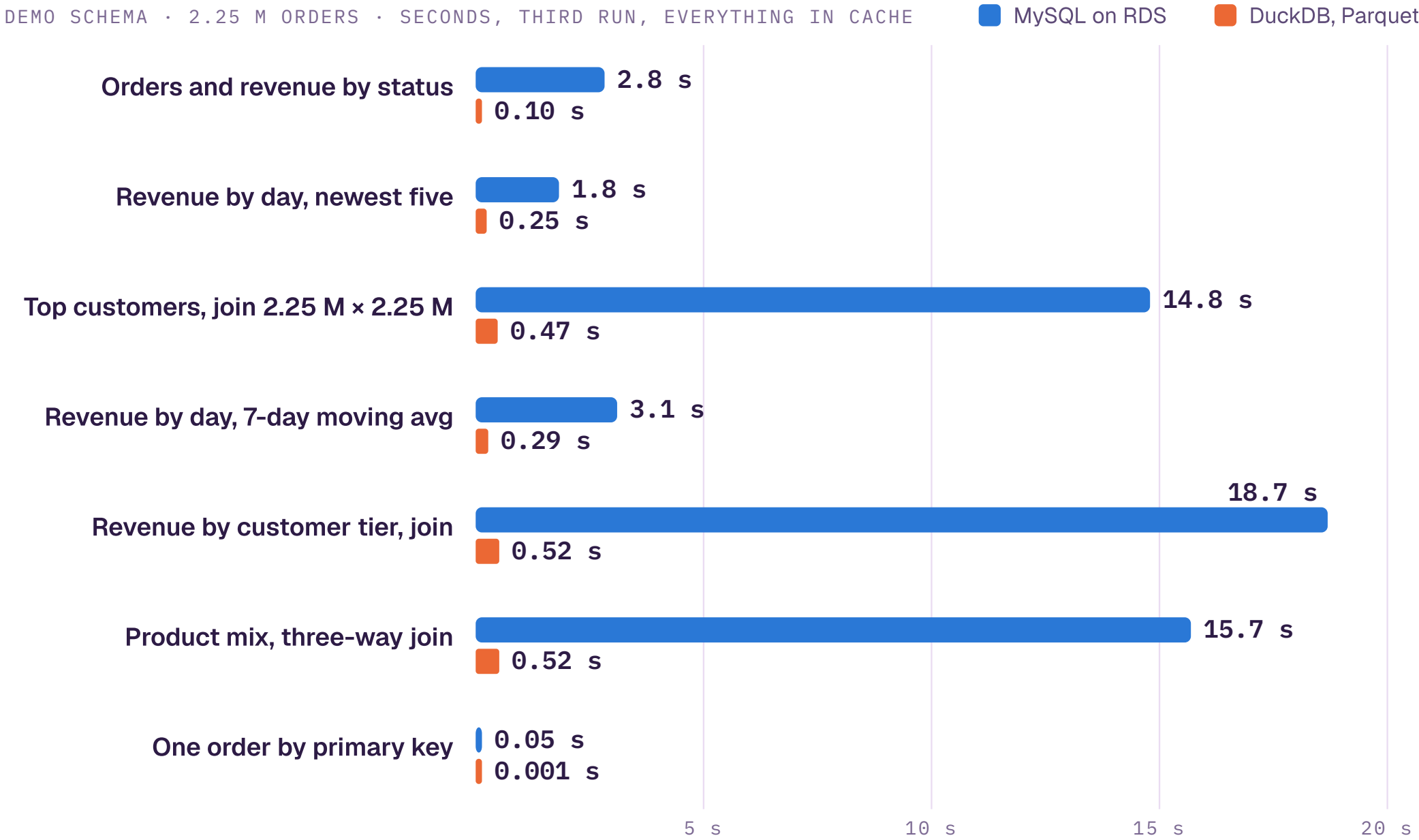
The first two are why you install it. The last two come with the copy.

SAME QUESTION, TWO ENGINES

Reports run on the copy.

Transactions stay on InnoDB, where a lookup by key is 55 ms. Reports go to DuckDB over the snapshot, and stop competing with your primary for the buffer pool.

Nothing was replicated for this. The files already existed for recovery.

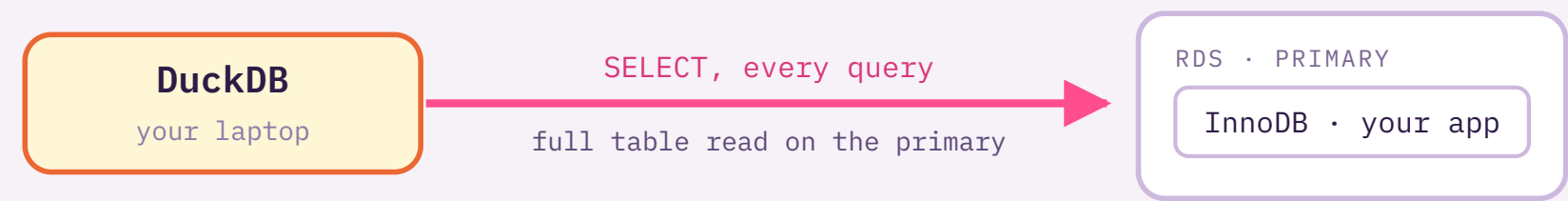


RDS MySQL 8.4.10 on a db.t4g.medium: 1.5 GB buffer pool, the whole demo schema (1.4 GB) in memory, third of three runs, all three within 5 %. DuckDB on the dbtrail box, an m7g.large with 2 vCPU, over the local snapshot files. Measured 10 Sep.

THE FIRST QUESTION AN ENGINEER ASKS

Why not just ATTACH?

ATTACH is a DuckDB feature: it mounts your MySQL as a schema, so `prod.orders` is one more DuckDB table, read from MySQL on every query.



```
D ATTACH 'host=rds...' AS prod (TYPE mysql);
D SELECT ... FROM prod.orders; -- reads RDS, now
```

- ✗ **Every query competes with your app**
"read directly from MySQL at query time": same buffer pool, same CPU.
- ✗ **The copy the docs suggest is by hand**
A full scan, stale when it finishes, redone from production.
- ✗ **No yesterday**
One state, the one you copied.

dbtrail: copy once, then only the binlog. DuckDB reads Parquet in your bucket and never talks to MySQL.



```
15:40 fold 0 SELECT on your tables
D SELECT ... FROM state_demo_orders; 0.5 s
```

- ✓ **The primary only ships its binlog**
What it already does for any replica. No contention with your app.
- ✓ **Fresh every 5 minutes, by folding**
Never a dump. Untouched tables carry forward.
- ✓ **Every version of every row**
The state at 14:03, the SQL to undo 14:04.

Small tables, occasional queries, no history needed: use the extension. It is free and good at that.

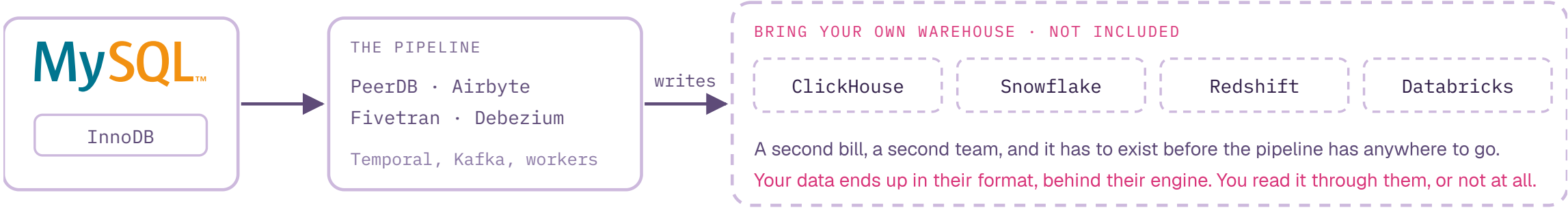
"HAVE YOU LOOKED AT PEERDB?"

Pipelines need a warehouse. This is the whole thing.

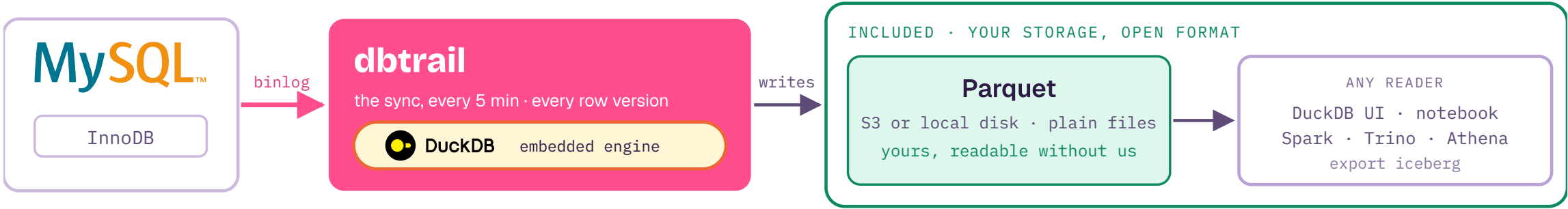
PeerDB, Airbyte, Fivetran, Debezium move your changes **into a warehouse you already run**. No ClickHouse, Snowflake, Redshift or Databricks: nothing to write to.

dbtrail is the sync, the storage and the engine in one: files in your bucket, DuckDB on top. **Nothing else to buy.**

CDC PIPELINES · PEERDB, AIRBYTE, FIVETRAN, DEBEZIUM



DBTRAIL · THE WHOLE THING



NO WAREHOUSE TO BUY FIRST · NO PIPELINE TO OPERATE · ONE BINARY, ONE BUCKET

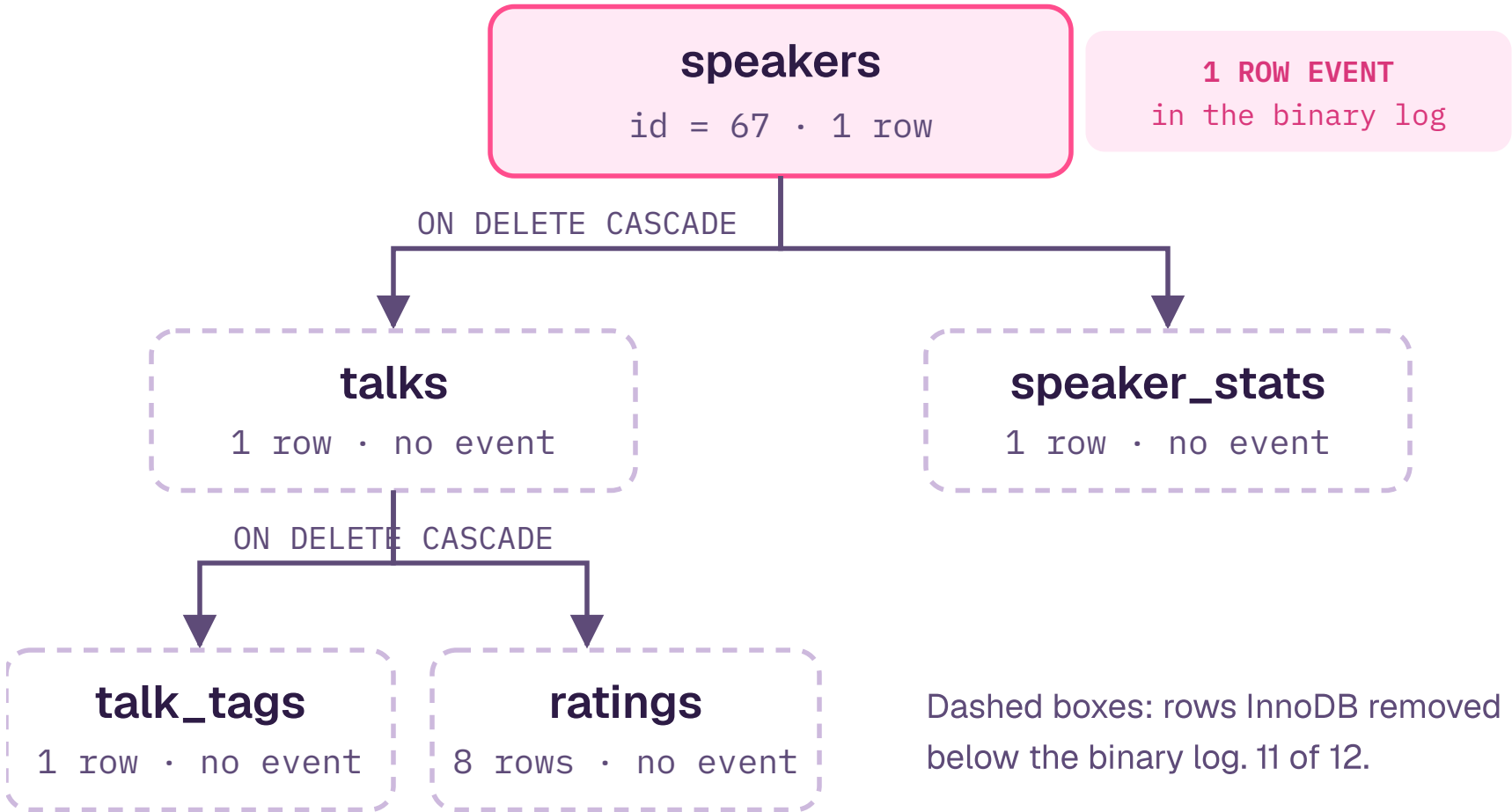
PEERDB WINS ON SOURCES AND LAG. SAID HERE ON PURPOSE.

LIVE RDS, ONE STATEMENT

One row, twelve gone.

```
mysql> DELETE FROM speakers WHERE id = 67;  
Query OK, 1 row affected (0.01 sec)
```

MySQL told the client	1
Rows that left the database	12
Row events in the binary log	1
Statements needed to undo it	12



MYSQL BUG #32506 · TRUE THROUGH 8.4 AND ON MARIADB · FIXED IN 9.6

What did this row hold before they touched it?

WITHOUT DBTRAIL

9

steps, and only if the log survived

- check binlog_format and row_image
- find which file covers that hour
- pull the logs off the host
- find the position, decode, grep
- map @1 @2 @3 to column names by hand
- ~~xtrabackup stops at step 1 on RDS~~

WITH DBTRAIL

1

step, 1.2 s, before and after image

- ask the console, or ask Claude

A restore to a scratch server is 10 steps and 38 to 42 seconds, and it hands you the backup's moment, not the hour you asked about. Measured 8 Sep on RDS MySQL 8.4.

Eleven of these have no event id.

Look at the **[0]**. Those rows were never written to the binary log, so there is no event to reverse. They are rebuilt from the children's own inserts and the foreign key.

The parent is the only one with a real id and a GTID.

```
-- [0] talk_tags      pk=8
-- [0] ratings        pk=90 .. 97    (eight)
-- [0] talks           pk=8
-- [0] speaker_stats  pk=67
-- [18036220] speakers pk=67
--      gtid=3ca0f1c0-...:195227474
```

```
INSERT INTO talks (conf_day, id, room, slot,
                  speaker_id, title, type)
VALUES (11, 8, 'Matterhorn 1', '9:00am', 67,
        'Fireside Chat, the Columnstore Revolution',
        'Keynote');
```

12 statements, 1 parent and 11 children, generated on stage from the console.
Never applied for you.

AS OF, in your client.

It speaks the MySQL wire protocol. Use your own client, pointed at the time travel port.

Timestamps are UTC. Run date -u before you pick one.

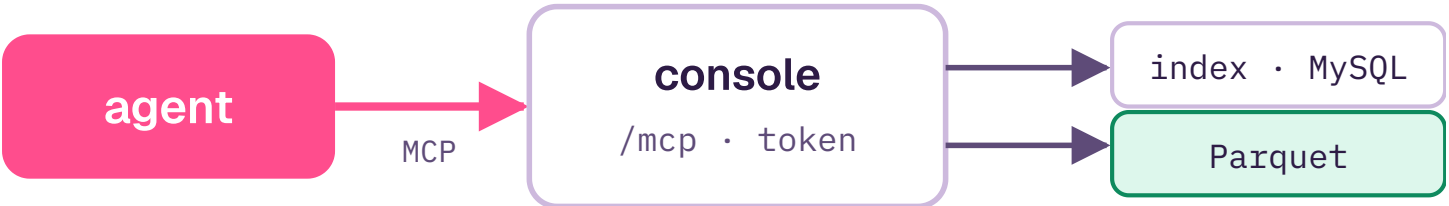
```
mysql> SELECT * FROM speakers WHERE id = 67;
Empty set (0.00 sec)

mysql> SELECT * FROM speakers WHERE id = 67
-> AS OF '2026-08-19 09:12:00';

+----+-----+-----+-----+
| id | name          | role   | company |
+----+-----+-----+-----+
| 67 | Peter Zaitsev | Founder | Percona |
+----+-----+-----+-----+
```

Ask it in English.

The console is an **MCP server**. Claude Desktop, Cursor or your own agent get six read-only tools: **query**, **recover**, **recover_cascade**, **reconstruct**, **status**, **list_schema_changes**. Ask in English, get the history and the SQL back.



RECOVER RETURNS SQL. NOTHING HERE EXECUTES IT.

someone deleted the user peter zaitsev please restore it. i believe there are FKs involved

Thought for 2s Restoring a deleted database user's account.

I'll look for the deletion in the binlog index first, then generate the reversal SQL including any FK cascades.

Used dbtrail integration, loaded tools >

Found it. The row was in `perconalive.speakers`, id 67, deleted today at 09:35:34 UTC. The cascade hit 11 child rows across four tables. Here is the reversal script dbtrail generated (dry-run, nothing has been applied; I have no write path to your server, so you run this):

```
sql
SET FOREIGN_KEY_CHECKS=0;
BEGIN;
SET time_zone = '+00:00';
SET sql_mode = 'STRICT_TRANS_TABLES,NO_ENGINE_SUBSTITUTION';

INSERT INTO `perconalive`.`talk_tags` (`id`, `tag`, `talk_id`) VALUES (8, 'General', 8

INSERT INTO `perconalive`.`ratings` (`attendee`, `id`, `score`, `talk_id`) VALUES ('at
INSERT INTO `perconalive`.`ratings` (`attendee`, `id`, `score`, `talk_id`) VALUES ('at
INSERT INTO `perconalive`.`ratings` (`attendee`, `id`, `score`, `talk_id`) VALUES ('at
INSERT INTO `perconalive`.`ratings` (`attendee`, `id`, `score`, `talk_id`) VALUES ('at
INSERT INTO `perconalive`.`ratings` (`attendee`, `id`, `score`, `talk_id`) VALUES ('at
INSERT INTO `perconalive`.`ratings` (`attendee`, `id`, `score`, `talk_id`) VALUES ('at
INSERT INTO `perconalive`.`ratings` (`attendee`, `id`, `score`, `talk_id`) VALUES ('at
INSERT INTO `perconalive`.`ratings` (`attendee`, `id`, `score`, `talk_id`) VALUES ('at
INSERT INTO `perconalive`.`ratings` (`attendee`, `id`, `score`, `talk_id`) VALUES ('at
INSERT INTO `perconalive`.`ratings` (`attendee`, `id`, `score`, `talk_id`) VALUES ('at
INSERT INTO `perconalive`.`ratings` (`attendee`, `id`, `score`, `talk_id`) VALUES ('at
```

A real run, this morning, against the conference schema. Dry run: 12 rows back, and it says what it is confident about and what it is not.

THE OBJECTION YOU ARE ABOUT TO MAKE

My schema changes.

A binlog row image is a list of values with **no names attached**. Add a column in the middle and everything after it shifts one place.

WHEN THE ROW WAS DELETED

id	name	role	company
----	------	------	---------

AFTER ALTER ... ADD COLUMN COUNTRY AFTER NAME

id	name	country	role	company
----	------	---------	------	---------

```
mysql> ALTER TABLE speakers
->   ADD COLUMN country VARCHAR(2)
->   NOT NULL DEFAULT 'NL' AFTER name;

-- what dbtrail generates to put him back
INSERT INTO `speakers` (`company`,`id`,`name`,`role`)
VALUES ('Percona',67,'Peter Zaitsev','Founder');
```

It names its columns. country is absent, so the server fills the default. Positions never reach the SQL. Every event is decoded against the schema that was in effect the second it happened.

Reversing row events is an old, good idea.

MyUndelete

READS THE LOG BACKWARDS

"the relevant binary logs must still exist"



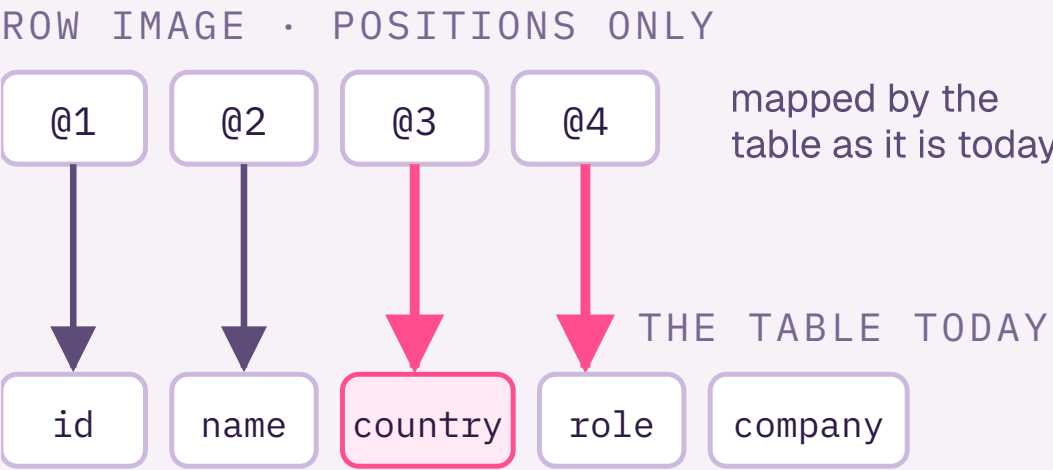
the row someone wants back

DBTRAIL
The index keeps its own copy of every row event. Purge whatever you like.

A MySQL component

SAME IDEA, IN THE SERVER

"the tables must still exist with compatible definitions"



@3 lands in country, @4 in role. Both wrong.

DBTRAIL
A schema snapshot per point in time. Every event decodes against the table as it was that second.

gtid_info plugin, MariaDB

JULY 2026 · NAMES ITS LIMITS

the condition nobody lists: cascades



Perfect binlog reading still returns 1 row of 12.

DBTRAIL
Children are rebuilt from their own inserts and the foreign key, as you saw on stage.

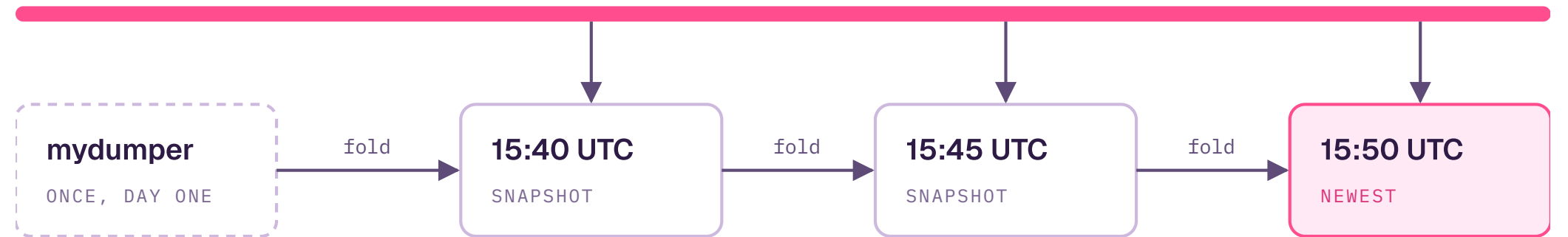
Quotes from lefred.be, "MariaDB GTID Info & Flashback Plugin", 23 July 2026. He says himself it is not a replacement for backups.

HOW FRESH IS THE COPY

Five minutes behind. Zero reads on the primary.

- A fold: newest snapshot + changes since = **next snapshot**
- mydumper ran **once**, the first night
- Untouched tables are carried forward, not rewritten
- Snapshots **check themselves**: previous one folded forward, compared with the new one

RECORDING, NEVER STOPS

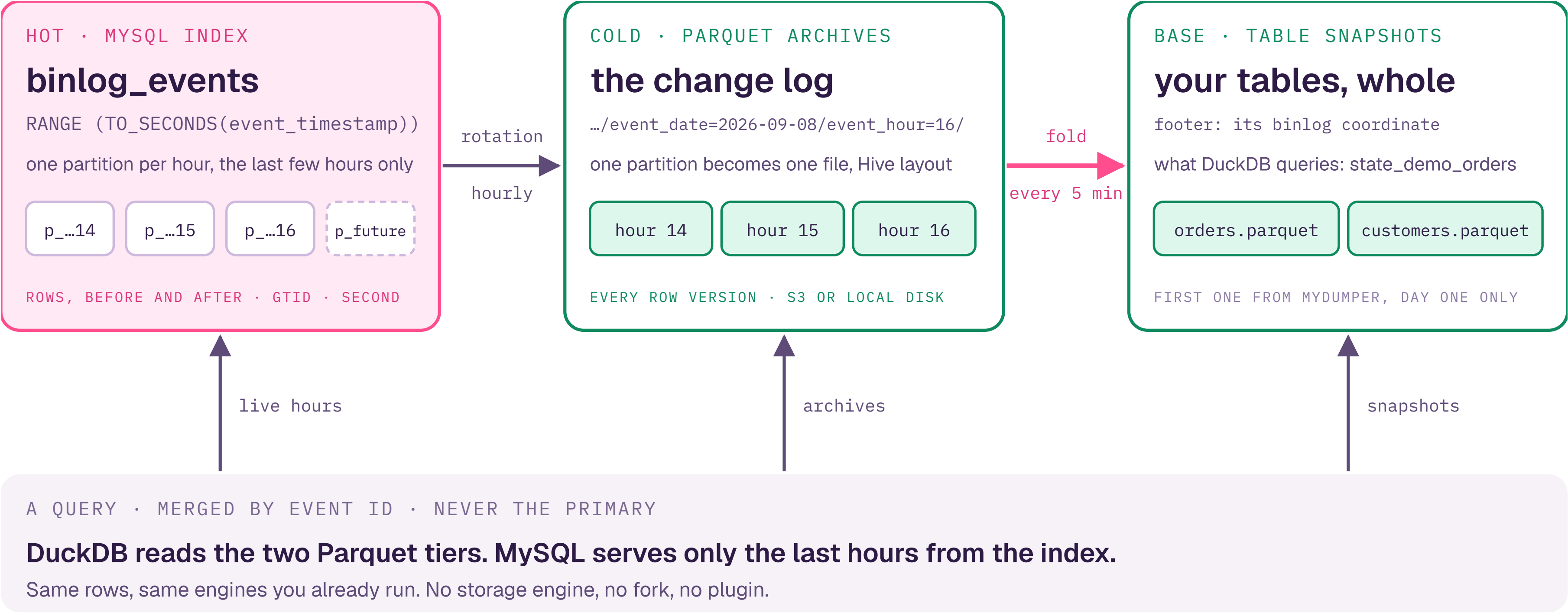


EVERY FOLD READS 0 ROWS FROM MYSQL

```
D SELECT getvariable('bintrail_newest_snapshot');
s3://.../baselinesabirds/2026-09-11T15-50-00Z/
D SELECT status, count(*) FROM state_demo_orders
  GROUP BY 1;
pending 451055 · shipped 450401 · delivered 449709 · ...
```

Set on the Backups page: every 5m. The nightly full dump of 8 tables and 2.2 M rows each read the RDS in full. The fold reads none of it.

Three tiers, no new engine.



NOTHING INSTALLED IN YOUR DATABASE · THE ARCHIVE STAYS PLAIN PARQUET ON PURPOSE · OPEN IT WITH ANYTHING



Open it with DuckDB.

The snapshot the fold just wrote is a directory of Parquet files. **duckdb -ui** over it: one view per table, and the footer of every file names the binlog position it stands on.

Every five minutes the views move to the newest snapshot. Nothing on this page talks to MySQL.

lake

▶

▼

```
1  WITH snap AS (  
2      SELECT timezone('UTC', strtptime(  
3          regexp_extract(max(file), '/(\d{4}-\d{2}-\d{2}T\d{2}-\d{2}-\d{2})Z/', 1),  
4          '%Y-%m-%dT%H-%M-%S')) AS folded_at  
5      FROM glob('/opt/duckdb-ui/baselines/abirds/20*/_SUCCESS')  
6  )  
7  SELECT folded_at AS snapshot_amsterdam,  
8         now() AS now_amsterdam,  
9         now() - folded_at AS snapshot_age,  
10        (SELECT max(created_at) FROM state_demo_orders) AS newest_order,  
11        (SELECT count(*) FROM state_demo_orders  
12         WHERE created_at >= folded_at - INTERVAL 5 MINUTE) AS orders_in_last_5_min_of_snapshot,  
13        (SELECT count(*) FROM state_perconalive_speakers) AS speakers,  
14        (SELECT count(*) FROM state_perconalive_ratings) AS ratings  
15  FROM snap;
```

1 row returned in 386ms ⓘ

	snapshot_amsterdam	now_amsterdam	snapshot_age	newest_order	123 orders_in_last_5_min_of_snap
1	2026-09-10 07:00:14+00	2026-09-10 07:07:15.987894+00	00:07:01.987894	2026-09-10 07:00:10+00	

📄

⬇

▼

lake

▶

▼

```
1  SELECT key::VARCHAR AS key, value::VARCHAR AS value  
2  FROM parquet_kv_metadata('/opt/duckdb-ui/baselines/abirds/current/perconalive/speakers.parquet')  
3  WHERE key::VARCHAR LIKE 'bintrail.%' AND key::VARCHAR <> 'bintrail.create_table_sql';
```

9 rows returned in 353ms ⓘ

	T key	T value	
1	bintrail.baseline_binlog_file	mysql-bin-changelog.050673	
2	bintrail.baseline_binlog_position	107701	
3	bintrail.bintrail_version	0.1.0	
4	bintrail.derived from path	s3://mv-bintrail-archives/baselinesabirds/202...	

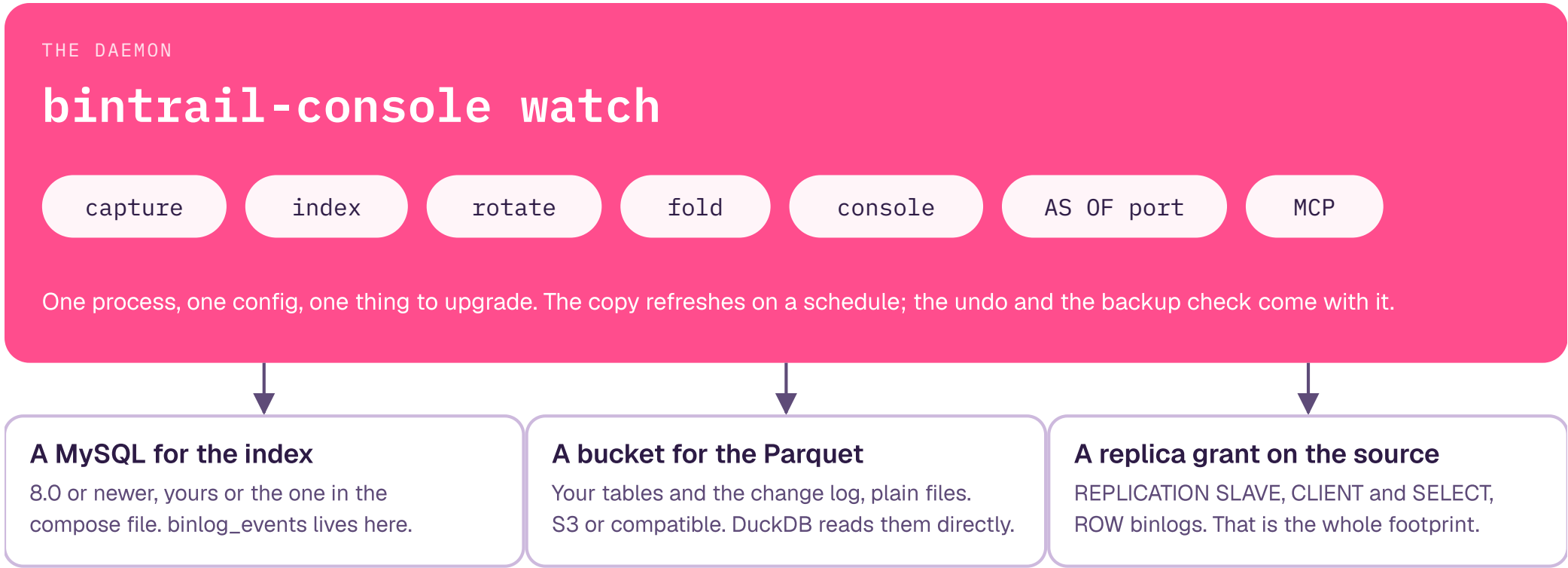
WHAT YOU ACTUALLY RUN

One binary. Nothing else to operate.

The copy on these slides is **one daemon**: it connects the way a replica does, folds the binlog into Parquet every five minutes, and serves the console, the AS OF port and MCP. DuckDB reads the files.

No warehouse. No pipeline. No Temporal, no Kafka, no second MySQL to patch.

Apache 2.0. Linux amd64 and arm64: tarball, deb, rpm, Docker image, and a compose file that ships the index MySQL. github.com/dbtrail/dbtrail



NO AGENT · NO PLUGIN · NO FORK · NO WAREHOUSE · NOTHING TO INSTALL IN THE SOURCE

WHAT IT IS NOT

Where it stops.

AVAILABILITY

Not high availability

The app never points here. Nothing fails over.

BACKUP

Not your backup

Starts the day you install it. Keep the physical one; this checks it.

LAG

Minutes behind, not seconds

Your setting. ROW binlog with full images is the price of entry.

OPERATIONS

Yours to operate

Index and disk, like one more replica. One source per index.

FORMAT

Not a lakehouse

No catalog, no Iceberg storage. Parquet of your MySQL, in your bucket.

Said in the same voice as the strengths. A copy you can trust is one whose edges you know.

TAKE IT HOME

Apache 2.0. All of it.

Capture, index, console, recovery, the Parquet you open in DuckDB, and the MCP server that lets an assistant drive it. Every command in this talk runs on the free binary.

github.com/dbtrail/dbtrail

Daniel Guzman Burgos Questions welcome, and so are the hard ones

github.com/dbtrail/dbtrail

READMEContributingApache-2.0 licenseSecurity

 **dbtrail**

The open-source time-travel flashback for MySQL and Postgres.

reads your binlog/wal → streams indexed events → access your previous data



See every row change in MySQL and PostgreSQL, and undo the ones you didn't want. No restore, no locks, no schema changes.

releasev0.78.0

licenseApache-2.0

```
SELECT * FROM orders WHERE id = 123 AS OF '2026-05-20 14:00:00'
```